

TERRAFORM LESSON

Francisco Javier Moreno Diaz
fmorenod@gmail.com

Agenda

- IaC
- Terraform
- Lifecycle (and its commands)
- Blocks: Providers, Resource, Data, etc.
- Setup
- Exercise

What is IaC ?

Infrastructure as Code (IaC) is the process of managing and provisioning ~~cloud~~ infrastructure with ~~machine-readable~~ definition files.

Think of it as executable documentation.

Provide a codified workflow to create infrastructure (including changes/updates – version and scale)

Integrate with application code workflows (Git, CI/CD tools)

Provide reusable (repeatable & auditable) modules for easy sharing and collaboration
Safely test changes using native-tools of IaC or 3rd party

Enforce security policy and organizational standards

To sum all, it offers **speed**, **reliability**, **collaboration and safety** (rollback, depens on inter)



What is Terraform?

HashiCorp
an IBM Company



HashiCorp
Terraform

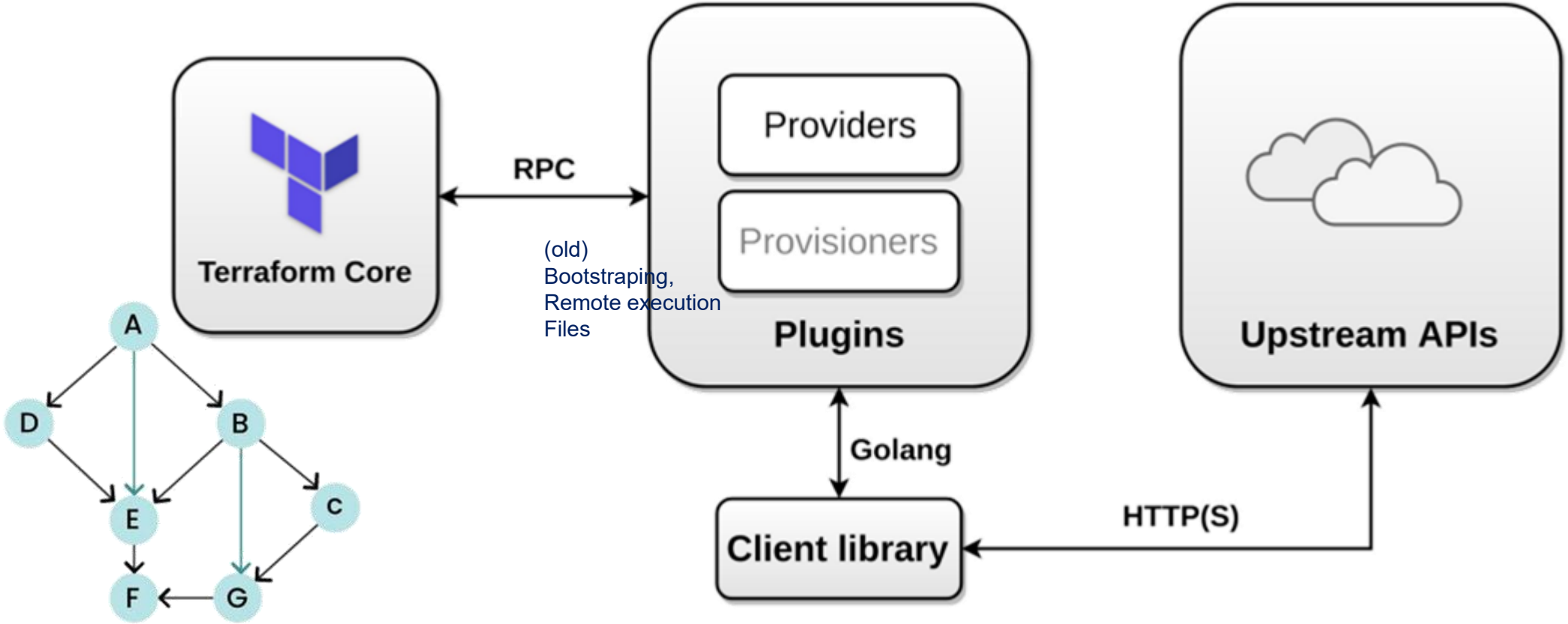
Declarative Open-source Provisioning tool (IaC).

It ships as a cross-platform (Win/Lin/Mac) single binary which is written in Go.

Executable Documentation: Human and machine readable.
Easy to learn.
Test, share, re-use, automate.
Works on all major cloud providers.



Architecture



Terraform Core is like the game console with multiple ports, and the Providers is like the cartridge (you can change or add more)



Comparison with others tools

Feature / Criteria	Terraform	Ansible / Puppet	Pulumi	Bash / Python / AWS CLI
Primary Category	Infrastructure Provisioning (Provisioning).	Configuration Management (Config Management).	Infrastructure Provisioning (Provisioning).	Imperative Scripting
Language / Syntax	HCL (Declarative DSL).	YAML / Proprietary DSL (Procedural/Declarative).	General Languages (TypeScript, Python, Go).	Any Language for scripting, i.e. Ruby, Python, Perl, .etc., or command line
Infrastructure State	Immutable (replace resources).	Mutable (in-place updates on live OS).	Immutable (replace resources).	None; scripts must manually check if resources exist
State Tracking	Explicit state file (.tfstate) mapped to cloud.	Usually stateless (queries live system state).	Explicit state file (managed via Pulumi Service/Backend).	Difficult to achieve; re-running often creates duplicates or errors.
Learning Curve	Low/Medium (DSL focused strictly on infrastructure).	Low (Simple YAML structure).	High (Requires full software engineering & OOP skills).	High (Requires full software engineering & OOP skills).

What is the state file and its risk

What is it ?

It’s the map of created, or imported, resources to the code.

Where is located?

Can be local, remote (on your account) or using Terraform Cloud (Licensed).

What happened if its corrupted or deleted ?

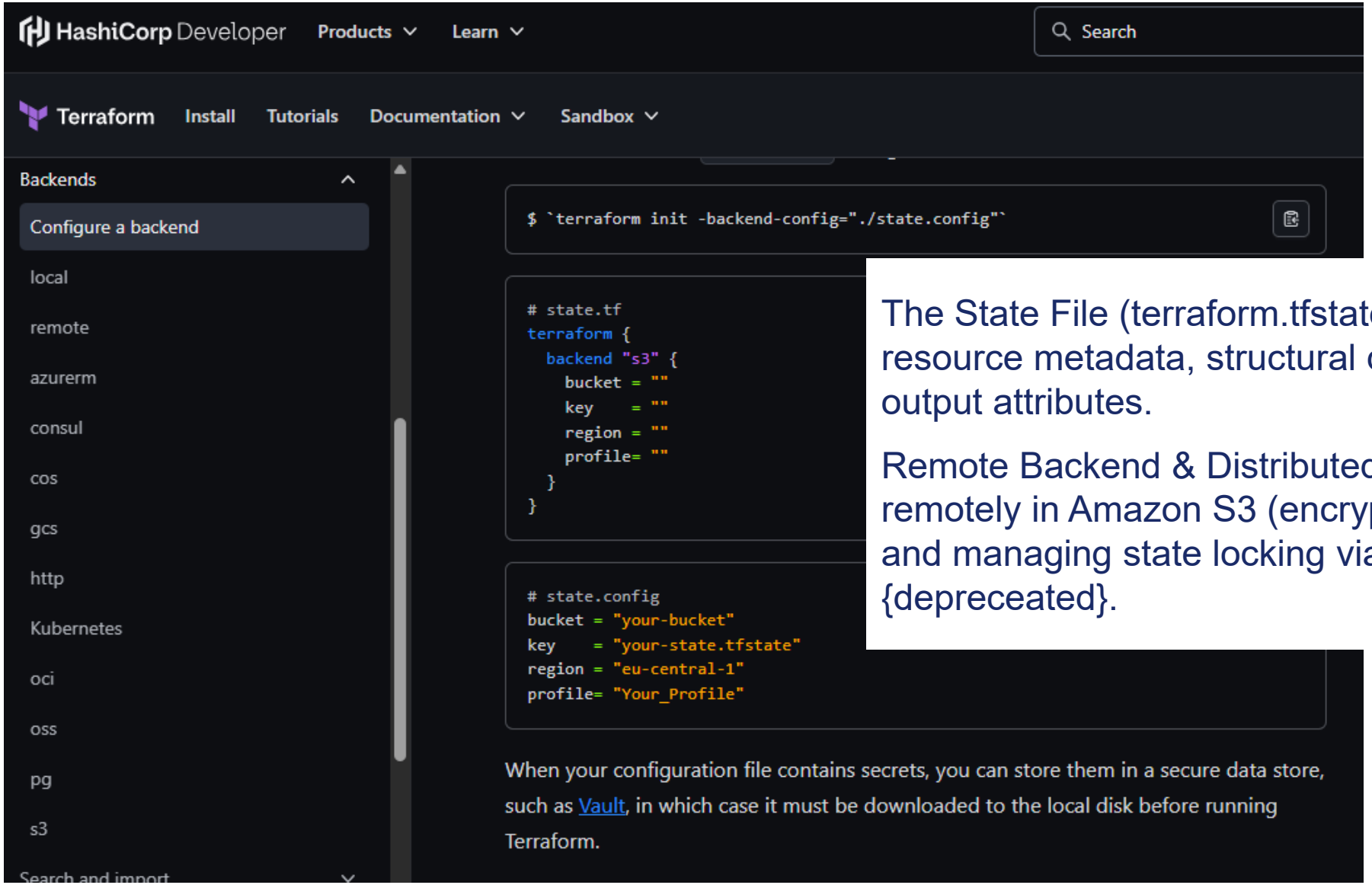
You lost your traced resources, when you launched it deployed new resources and can be conflicted with the previous one; or it can be blocked.

When happened with multiple teams like to deploy ?

It can be corrupted due to multiple requests, you have to lock the state file



State file and our demo



The State File (terraform.tfstate): JSON structure storing resource metadata, structural dependencies, and sensitive output attributes.

Remote Backend & Distributed Locking: Storing state remotely in Amazon S3 (encrypted at rest via AES-256) and managing state locking via Amazon DynamoDB {depreceated}.

Local vs Remote Backend

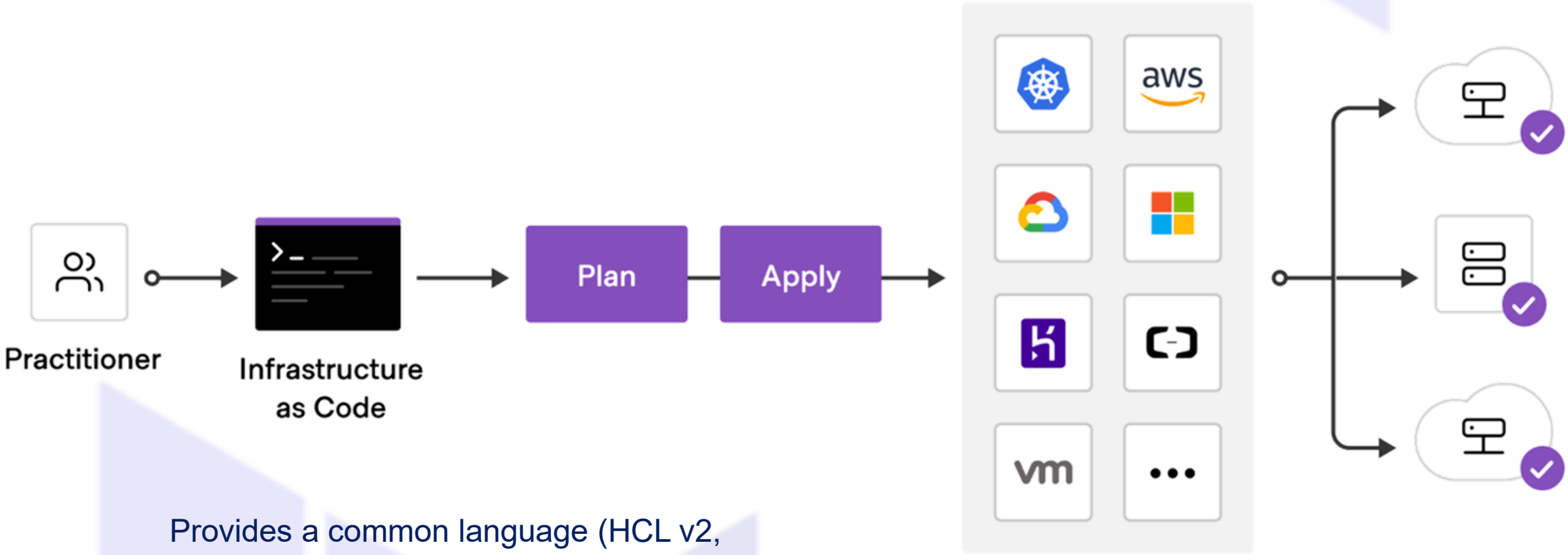
Capability / Risk	[Default] Local State Storage (local)	Remote Backend (s3 + dynamodb)
Team Collaboration	Isolated to a single developer machine; prone to merge conflicts.	Centralized single source of truth for distributed teams.
State Locking	Supported locally per command execution only.	Distributed locking viaObject Storage or DynamoDB, which prevents concurrent race conditions.
Security & Secrets	Secrets stored in plain text on local developer disk.	Server-side encryption (AES-256) and IAM access policies.
Durability	High risk of data loss due to local disk failure.	High availability, S3 object versioning, and automated backups.

To config and store a remote backend, we will use a combination of backend and lock explicit. You can configure on the TF code and insert your credentials on backend config on *terraform init* command.

To modern collaboration, you need a lock system to block corrupted state file. To do it explicit, we use dynamodb locking although it is deprecated.

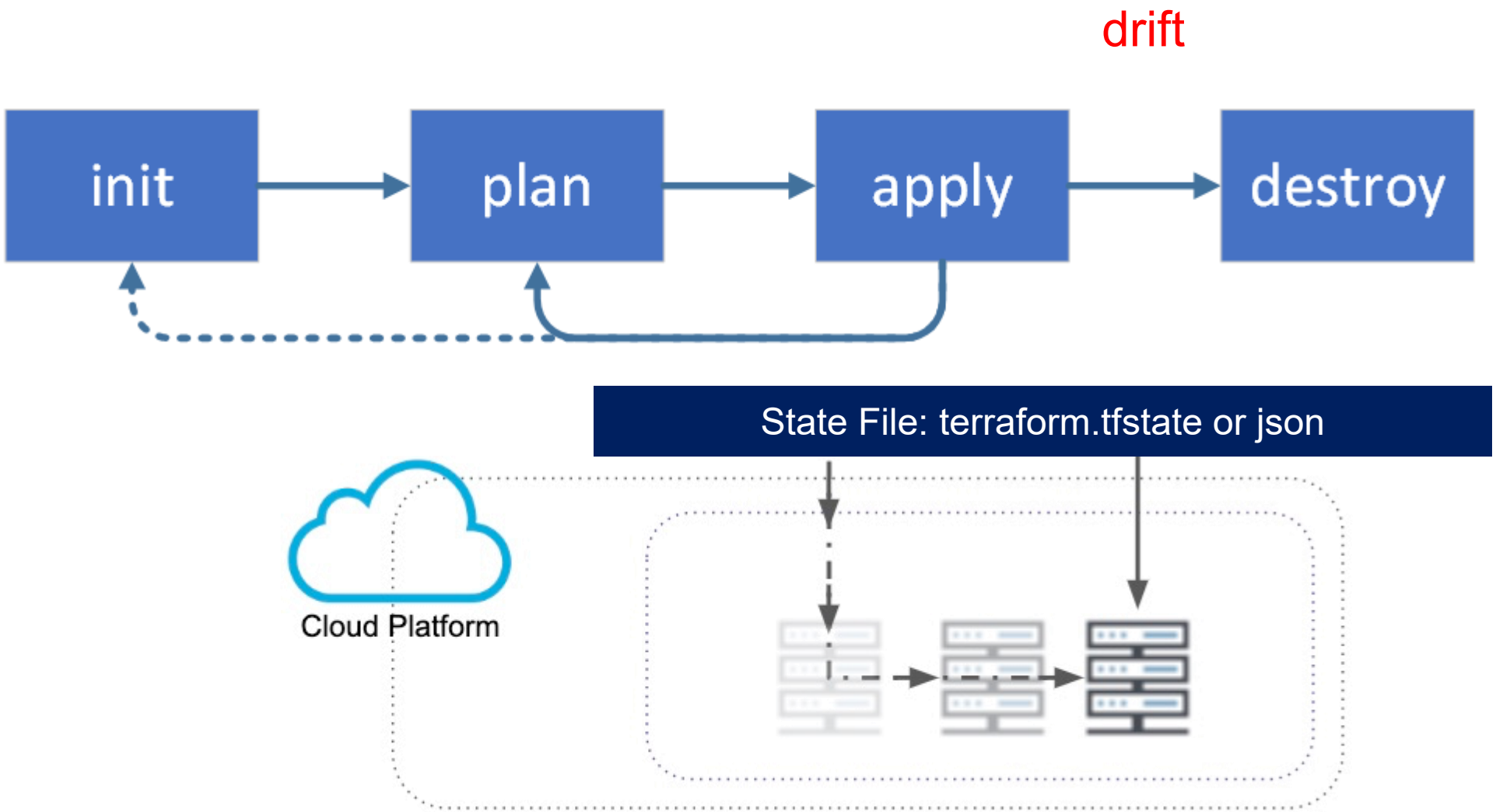
Use case

HashiCorp
Terraform



Provides a common language (HCL v2, HashiCorp Configuration Language) to orchestrate cloud resources and/or other providers.

Workflow as example



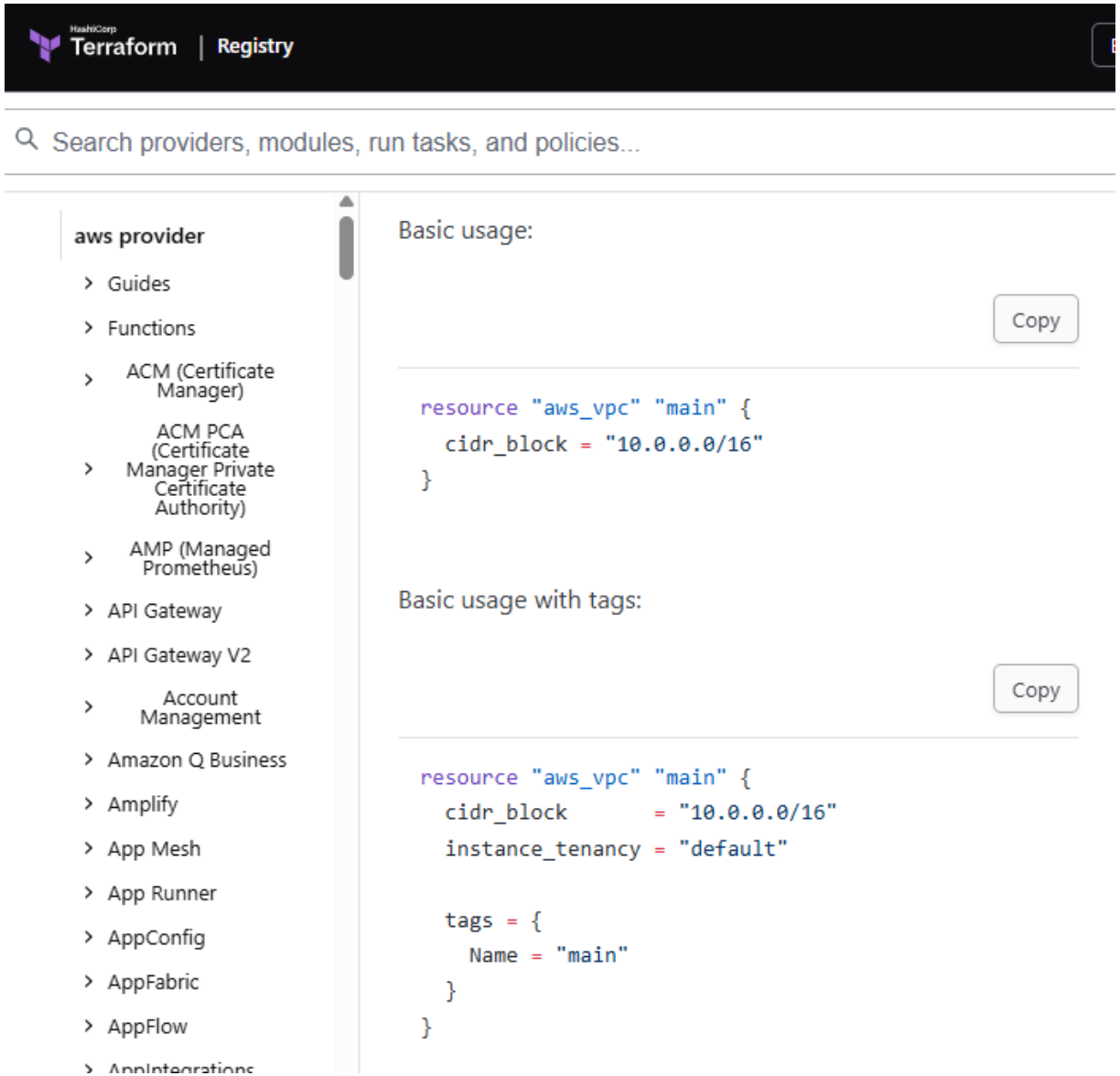
Terraform commands

is a command line tool. Terraform commands are either typed in manually or run automatically from a script. Terraform has subcommands that perform different actions.

```
terraform version
terraform help
terraform init [-backend-config=]
terraform plan
terraform apply [-auto-approve]
terraform destroy
terraform fmt [-recursive]
terraform validate
terraform import
```

Type `terraform subcommand help` to view help on a particular subcommand.

Terraform Code



Terraform code is based on the HCL2 toolkit. HCL stands for.

Terraform code, or simply `terraform` is a declarative language that is specifically designed for provisioning infrastructure on any cloud or platform.

Terraform init

```
$ terraform init

Initializing provider plugins...
- Checking for available provider plugins...
- Downloading plugin for provider "aws" (hashicorp/aws) 2.35.0...
...
 provider.aws: version = "~> 2.35"

Terraform has been successfully initialized!
```

Terraform fetches any required providers and modules and stores them in the `.terraform` directory. If you add, change or update your modules or providers you will need to run `terraform init` again.

Terraform plan

```
$ terraform plan
An execution plan has been generated and is shown below.
Terraform will perform the following actions:
  # aws_vpc.main will be created
  + resource "aws_vpc" "main" {
      + arn                              = (known after apply)
      + cidr_block                       = "10.0.0.0/16"
      ...
      + instance_tenancy                 = "dedicated"
    }
```

Preview your changes with `terraform plan` before you apply them. This step is now integrated with `terraform apply`.

Changing Existing Infrastructure

Whenever you run a `terraform plan` or `apply`, Terraform **reconciles** three different data sources:

1. What you wrote in your code
2. The state file
3. What actually exists

Terraform does its best to add, delete, change, or replace existing resources based on what is in your *.tf files. Here are the four different things that can happen to each resource during a plan/apply:

```
+   create
-   destroy
-/+ replace
~   update in-place
```

Files

terraform.state

STATE FILE

VARIABLES

terraform.tfvars

terraform-provider.tf

CODE

terraform- instances.tf





Depends on your extension, you reorganize the content on files, however it is a good advice to manage a similar structure

Blocks

Providers

Variables

Resource

Data

Outputs

Remote Backend

Out of scope for this lesson:

Modules

Provisioner

Workspace

Loops or Conditions

Built-in Variables

Terraform Provider Configuration

The terraform core program requires at least one provider to build anything. You can manually configure which version(s) of a provider you would like to use. If you leave this option out, Terraform will default to the latest available version of the provider.

```
provider "aws" {
  version = "=2.35.0"
}
```

```
- = (or no operator): exact version equality
- !=: version not equal
- \>, >=, <, <=: version comparison
- ~>: pessimistic constraint, constraining both the oldest and newest
version allowed. ~> 0.9 is equivalent to >= 0.9, < 1.0, and ~> 0.8.4
is equivalent to >= 0.8.4, < 0.9
```

Variables

INPUT

Define variables: See priority next slide.

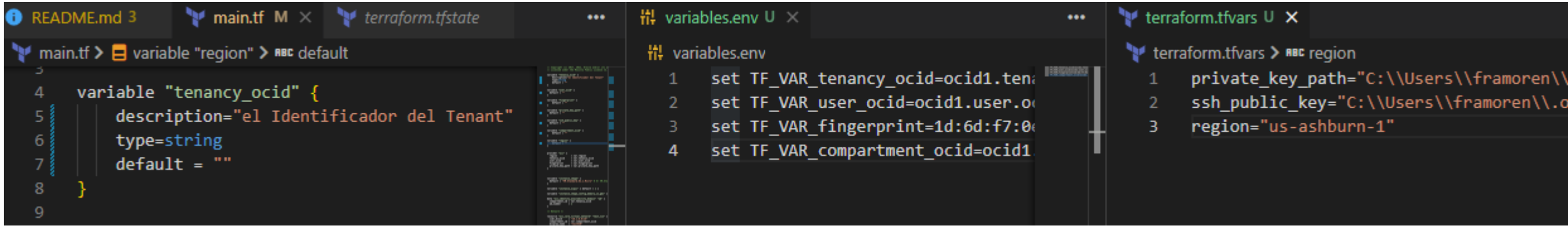
Types of variables:

* Simple: string, number, bool
* Constructed: list, set, map, object, tuple.
* Validation: regexp, null,

Sensitive << Don’t show on the UI

OUTPUT

Attributes: Sensitive << Although is stored on Remote Backend.



Variables

Terraform variables are placed in a file called variables.tf. Variables can have default settings. If you omit the default, the user will be prompted to enter a value. Here we are declaring the variables that we intend to use.

Priority (high to low)

1. Command line flag - run as a command line switch
2. Configuration file - set in your terraform. tfvars file
3. Environment variable - part of your shell environment
4. Default Config - default value in variables.tf
5. User manual entry - if not specified, prompt the user for entry

Block Structure

```
provider "oci" {
  region           = var.region
  tenancy_ocid     = var.tenancy_ocid
  user_ocid        = var.user_ocid
  fingerprint      = var.fingerprint
  private_key_path = var.private_key_path
}
# See https://docs.oracle.com/iaas/images/
data "oci_core_images" "test_images" {
  compartment_id           = var.compartment_ocid
  operating_system         = "Oracle Linux"
  operating_system_version = "8"
  shape                    = var.instance_shape
  sort_by                  = "TIMECREATED"
  sort_order               = "DESC"
}
/* Network */
resource "oci_core_virtual_network" "test_vcn" {
  cidr_block     = "10.1.0.0/16"
  compartment_id = var.compartment_ocid
  display_name   = "testVCN"
  dns_label      = "testvcn"
}
output "Public_IP_LoadBalanceador" {
  value = "http://${oci_load_balancer_load_balancer.free_lo
}
```

variable
provider
data
resource
output

Module
Folder with code and has similar input and output
Terraform accesses TF files in the current folder only, does not enter to nested folders

How to Provision an AWS Instance

Before we start, we'll need to gather some basic information including (but not limited to):

Instance Name, Operating System (Image), VM Size, Geographical Location (Region), Security Groups

```
resource aws_instance "web" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t2.micro"

  tags = {
    Name = "HelloWorld"
  }
}
```

Anatomy of a Resource

Every terraform resource is structured the same way.

resource = Top level keyword

type = Type of resource. Example: `aws_instance` .

name = Arbitrary name to refer to this resource. Used internally by terraform. This field cannot be a variable.

```
resource type "name" {
  parameter = "foo"
  parameter2 = "bar"
  list = ["one", "two", "three"]
}
```

Data Sources

```
data "aws_ami" "ubuntu" {
  most_recent = true
  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-trusty-14.04-amd64-server-*"]
  }
  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
  owners = ["099720109477"] # Canonical
}
```

Data sources are a way of querying a provider to return an existing resource, so that we can access its parameters for our own use.

Terraform Dependency Mapping

Terraform can automatically keep track of dependencies for you. Look at the two resources below. Note the highlighted lines in the `aws_instance` resource. This is how we tell one resource to refer to another in terraform.

You can use depends_on

```
resource aws_key_pair "my-keypair" {
  key_name   = "my-keypair"
  public_key = file(var.public_key)
}

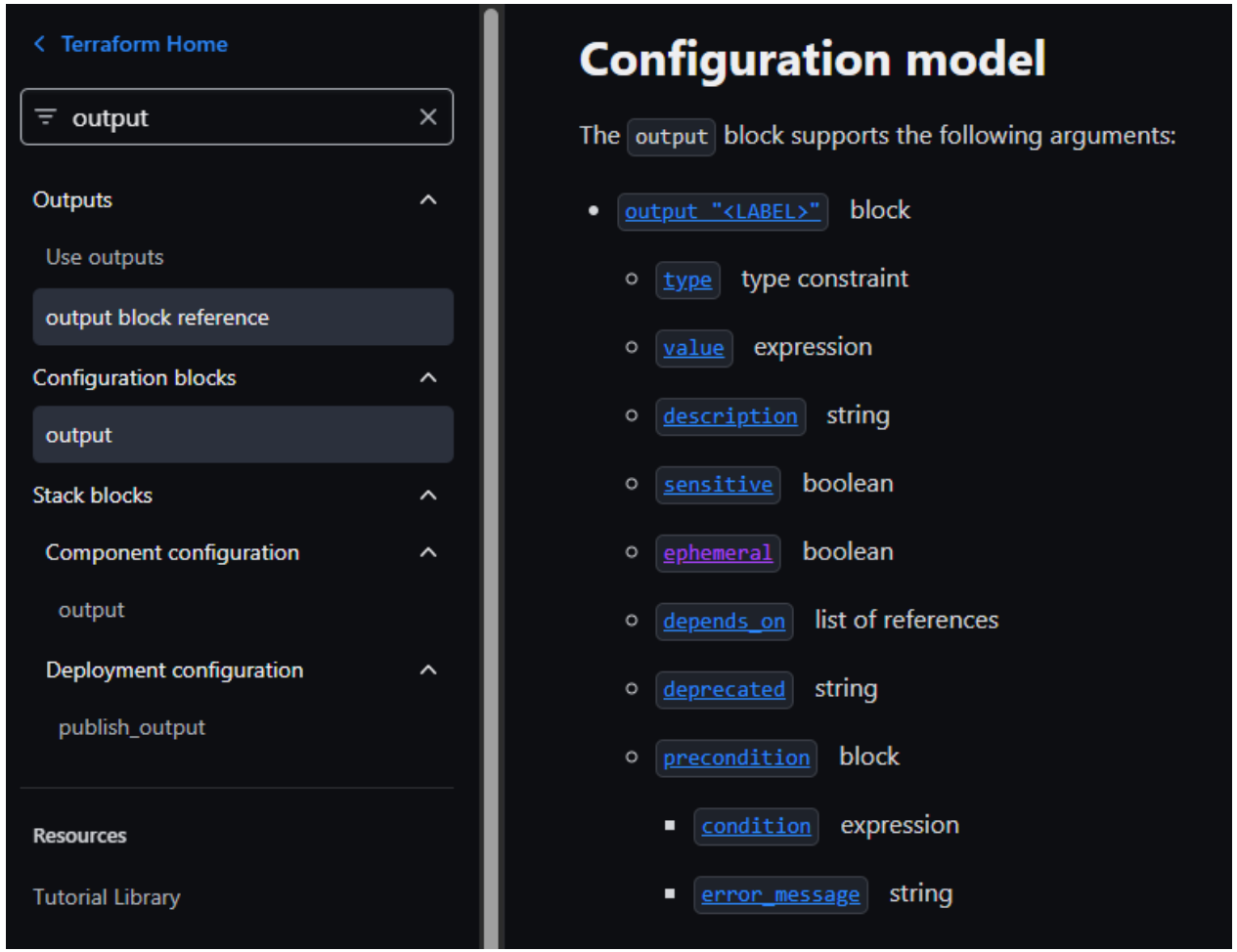
resource "aws_instance" "web" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t2.micro"
  key_name      = aws_key_pair.my-keypair.name
```

Terraform State

Terraform is a **stateful** application. This means that it keeps track of everything you build inside of a state file. You may have noticed the `terraform.tfstate` and `terraform.tfstate.backup` files that appeared inside your working directory. The state file is Terraform's source of record for everything it knows about.


```
{
  "terraform_version": "0.12.7",
  "serial": 14,
  "lineage": "452b4191-89f6-db17-a3b1-4470dcb00607",
  "outputs": {
    "catapp_url": {
      "value": "http://go-hashicat-5c0265179ccda553.workshop.aws.hashidemos.io",
      "type": "string"
    },
  }
}
```

Terraform Output



Mandatory for modules.

You need to review in an easy way, what it is the result of a deployment, no need all resource, you need the important.

i.e. You need the endpoint of ELB, no Launch Template, AMI Id, Subnets, VPC

Terraform Remote Backend

First you have to create:

Bucket S3: with Versioning, default encryption and global name.

DynamoDB: Partition Key as LockID as string.

You can make the change on code or using terraform init –backend-config=backend.tfvars

```
  # Backend Remoto con State Locking en DynamoDB
  backend "s3" {
    bucket         = "sandbox-tf-state-185163956096" # Nombre de tu bucket S3
preexistente
    key            = "master-lab/terraform.tfstate"   # Ruta del estado dentro del
bucket
    region         = "us-east-1"
    dynamodb_table = "sandbox-tf-locks"              # Tabla DynamoDB con Partition
Key LockID
    encrypt        = true                            # Cifrado en reposo habilitado
(AES-256)
  }
}
# ================================================================================
```

```
# Contenido de backend.tfvars
bucket         = "sandbox-tf-state-185163956096"
key            = "master-lab/terraform.tfstate"
region         = "us-east-1"
dynamodb_table = "sandbox-tf-locks"
encrypt        = true
```

Terraform Drift

To avoid manual configuration on the managed resources (bad practice), the system have to detect those changes and take action:

Feature / Aspect	terraform plan -refresh-only	terraform apply -refresh-only
Primary Purpose	To inspect and detect external changes (drift) between the real-world infrastructure and the current state file.	To update and synchronize the Terraform state file (terraform.tfstate) with the actual remote infrastructure.
Interactive Prompt	Displays the detected drift preview and exits cleanly without requesting confirmation.	Displays a proposed refresh plan and prompts for interactive user confirmation (yes) before saving to state.
Main Use Case	Auditing or monitoring drift in CI/CD pipelines or manually reviewing external changes safely.	Reconciling and accepting external infrastructure modifications as the new baseline in the state file.
Safety Level	Completely safe. Read-only operation with zero risk of altering state or infrastructure.	Safe alternative to terraform refresh. Replaced the legacy, non-interactive refresh command by offering approval controls before updating state.

Best Practices

- Keep configurations modular
- Use variables and workspaces for environments
- Never edit the state file manually
- Use `terraform fmt` & `terraform validate`
- Protect credentials (IAM roles, not keys)

Setup

1. Install of terraform and validate version
2. Install of AWS CLI and validation – optional-
3. Obtain AWS credentials for CLI and validate for AWS Academy' Sandbox Account
4. <Code by hand or ….>
5. Folder with Terraform Code
6. Lifecycle: Init, Plan and Apply
7. Check working environment
8. Changes and Lifecycle
9. End of infrastructure: Destroy

SDD o Vibe coding ?

Challenges using Sandbox Account

Reto 0:
Hacer un bucket de S3. Terraform apply and destroy.

Reto 1:
Crear una VPC que tenga un remote backend. Cargar en MS Forms: grupo, credenciales y nombre bucket.

Reto 2:
Seguir el diagrama. Tener en cuenta usar un loop en la creacion de las subnets y que las use el Terraform code.
Cargar en el Forms: grupo, credenciales, nombre bucket, y code snippet.

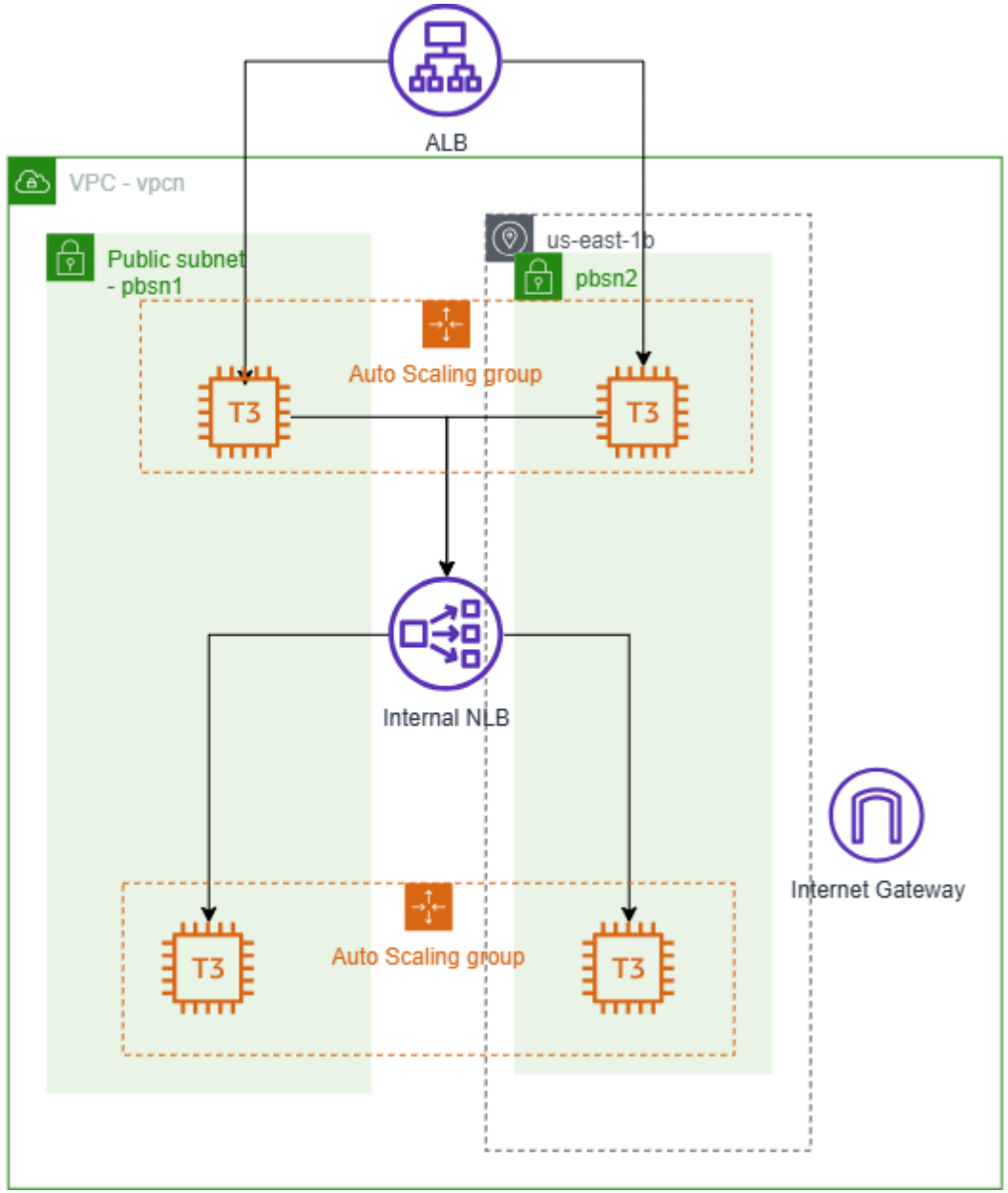
Reto 3:
Leer el codigo del Docker. Agregar la BD, comprobar la creacion, permisos, alcance.

Autoscaling group policy using stress
Simple Scaling Policy
Add actions using Cloudwatch

Docker
Ports and Images
80: fmorenod81/mtwa:web

`APP_ENDPOINT` = Internal NLB,
ie. `http://nlb_dns_name:8080`

Docker
Ports and Images
8080:
fmorenod81/mtwa:app



References & Resources

- https://developer.hashicorp.com/terraform/docs
- https://registry.terraform.io/providers/hashicorp/aws/latest/docs
- https://learn.hashicorp.com/collections/terraform/aws-get-started
- https://aws.amazon.com/free/